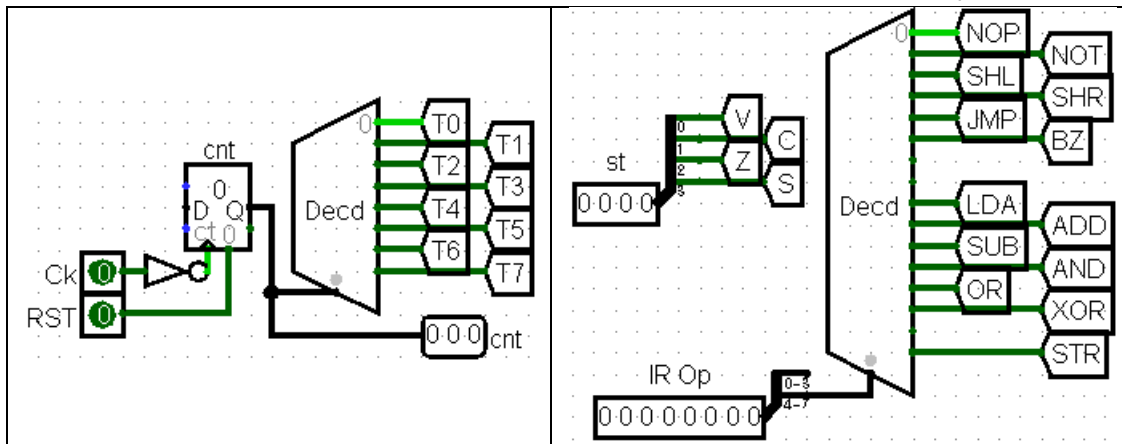


## 컴퓨터구조(2026 봄)숙제#08(1/2)

숙제#07의 제어 유닛을 수정하여, 전체 명령 중 4개의 명령(LDA, ADD, STR, JMP)을 수행하는 CPU의 제어 유닛을 구성하라. 1) 특정 명령이 특정 클럭에서 어떤 마이크로 연산을 수행하는지, 2) 특정 제어 비트가 어떤 마이크로 연산을 수행할 때 1이 되는지, 이 두 가지를 디지털 회로로 구성하면 된다. (\*.circ 파일 업로드)

모든 명령은 2바이트로 구성되며, 첫번째 바이트의 상위 4비트를 opcode로(하위 4비트는 무시), 두번째 바이트 전체를 operand (address)로 간주한다. Operand를 필요로 하지 않는 명령의 경우 두번째 바이트는 읽혀진 후 무시된다. 모든 명령의 수행은 4클럭(T0~T3)의 인출 사이클을 포함하여 8클럭(T0~T7) 걸린다고 간주하자.

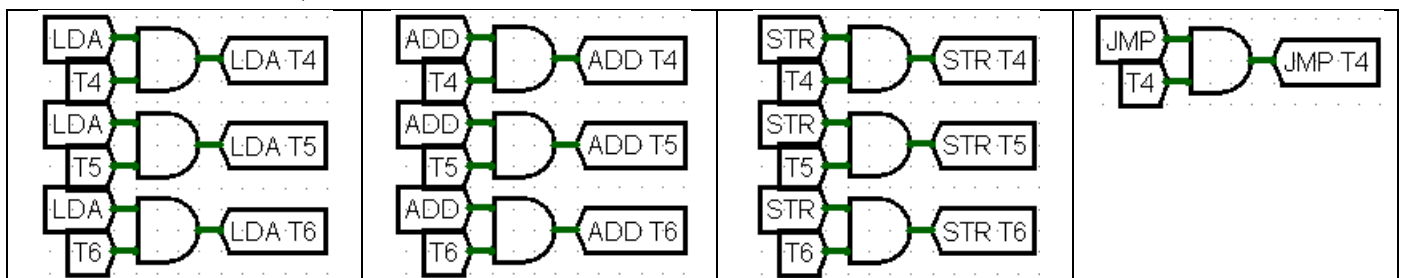
사이클 카운터를 T0부터 T7로 해독한 예, status 레지스터 입력, 그리고 IR Op의 상위 4비트를 해독한 예:



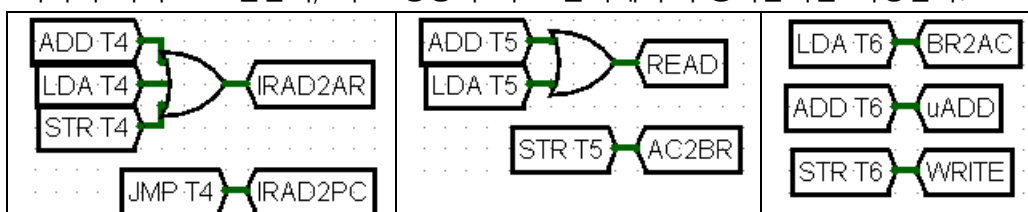
이번에 구현할 4개 명령의 실행 사이클의 각 (T4~T7)에서 수행할 마이크로 연산은 다음과 같다.(숙제#07)

| 명령                  | opcode   | 포맷       | T4      | T5    | T6    | T7   |
|---------------------|----------|----------|---------|-------|-------|------|
| Jump to addr        | 0x4 0100 | JMP addr | IRAD2PC | uNOP  | uNOP  | uNOP |
| Load M[addr] to AC  | 0x8 1000 | LDA addr | IRAD2AR | READ  | BR2AC | uNOP |
| Add M[addr] to AC   | 0x9 1001 | ADD addr | IRAD2AR | READ  | uADD  | uNOP |
| Store AC to M[addr] | 0xF 1111 | STR addr | IRAD2AR | AC2BR | WRITE | uNOP |

4개 명령의 각 클럭 중, uNOP 이외의 마이크로 연산을 수행하는 실행 클럭을 나타내는 “터널”을 만든다.



각각의 마이크로 연산이, 어느 명령의 어느 클럭에서 수행되는지를 지정한다.



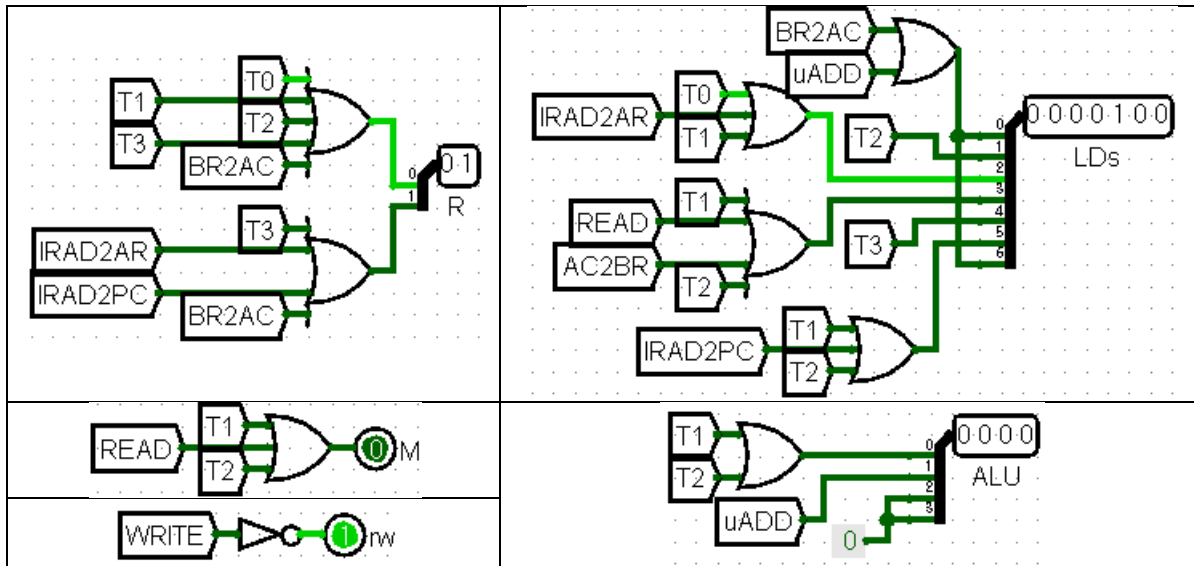
숙제#07의 마이크로 연산의 이름을 logisim에서 제공하는 “터널”의 이름으로 사용하면 편리하다.

## 컴퓨터구조(2026 봄)숙제#08(2/2)

다음은 각 마이크로 연산을 실행하는데 필요한 각 제어 비트 값의 집합 이다.(숙제#05)

|         | R  | M | ALU  | LDs  | rw  |                                           |
|---------|----|---|------|------|-----|-------------------------------------------|
| T0uOp   | 01 | x | 0000 | 0000 | 100 | 1 MAR <- PC                               |
| T1uOp   | 01 | 1 | 0001 | 0101 | 100 | 1 MBR <- M[MAR], (PC,MAR)<- PC+1          |
| T2uOp   | 01 | 1 | 0001 | 0101 | 010 | 1 IR_Op <- MBR, MBR <- M[MAR], PC <- PC+1 |
| T3uOp   | 11 | x | 0000 | 0010 | 000 | 1 IR_Addr <- MBR                          |
| uNOP    | xx | x | xxxx | 0000 | 000 | 1 NOP                                     |
| IRAD2AR | 10 | x | 0000 | 0000 | 100 | 1 MAR <- IR_Addr                          |
| READ    | xx | 1 | xxxx | 0001 | 000 | 1 MBR <- M[MAR]                           |
| WRITE   | xx | x | xxxx | 0000 | 000 | 0 M[MAR] <- MBR                           |
| AC2BR   | 00 | 0 | 0000 | 0001 | 000 | 1 MBR <- AC                               |
| BR2AC   | 11 | x | 0000 | 1000 | 001 | 1 AC <- MBR                               |
| uADD    | 00 | x | 0010 | 1000 | 001 | 1 AC <- AC + MBR                          |
| IRAD2PC | 10 | x | 0000 | 0100 | 000 | 1 PC <- IR_Addr                           |

제어 비트 별로 어떤 마이크로 연산에서 각 제어 비트 값이 1이 되는지를 회로로 구성한다.



다음과 같은 프로그램을 RAM의 0번지부터 입력한 후, reset입력을 1로 설정했다가 다시 0으로 설정하여 각 레지스터를 초기화 하고, "RAM sel" 신호를 1로 설정 한 후, "Simulate - Tick Frequency" 메뉴에서 적당한 클럭주파수(2~8Hz)를 선택하고, "Simulate - Tick enable(Ctrl-K)" 명령으로 클럭을 활성화 시켜서 실행시켜본다.

| 어셈블리어    | 기계어       |
|----------|-----------|
| LDA 0x0C | 0x80 0x0C |
| ADD 0x0D | 0x90 0x0D |
| STR 0x0C | 0xF0 0x0C |
| JMP 0x00 | 0x40 0x00 |

위 프로그램은 무한루프를 돌며 0x0C번지의 값을 0x0D 번지의 값 만큼 증가시키는 프로그램이다.

초기값으로, 0x0C번지에 0x00을, 0x0D번지에 0x01을 입력한 후 프로그램을 실행(클럭을 활성화)시켜서 0x0C번지의 값이 32클럭(8클럭x4명령어)마다 하나씩 증가하면 정상 동작하는 것이다.